

Concurrent Programming

Created by <Academia de Código_>

Introduction to Concurrency

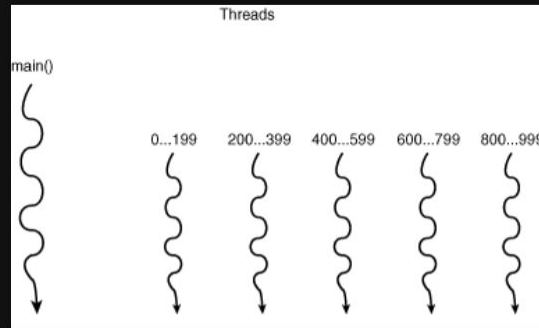
In the beginning... computers didn't have operating systems, they executed a single program from beginning to end with direct access to all the resources of the machine.

Operating Systems allow more than one program to execute, running individual programs in processes. Each process is isolated and executes independently from others, getting its share of resources such as memory, file handles, etc.

Processes can communicate through a variety of Inter Process Communication (IPC) mechanisms, such as sockets, signal handlers, shared memory, semaphores and files.

Threads

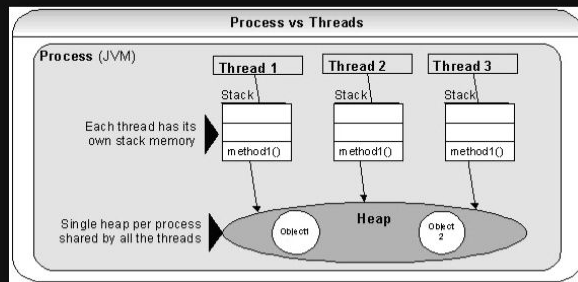
Allow multiple streams of program control flow to coexist and execute simultaneously within one single process.



They share process-wide resources such as memory and file handles, but they have their own stack and local variables.

Processes vs Threads

All Threads within a process have access to the same variables, allowing them to share data without using IPC mechanisms.



Without explicit synchronization to coordinate access to shared data, a thread may modify variables that another thread is in the middle of using, with unpredictable results

Why should I care?

Threads can simplify the development of complex systems, by turning complicated asynchronous code into simpler straight-line code.



Threads are the easiest way to take advantage of the computing power of multi-processor and multi-core systems.

Threads are an inescapable feature of the Java language, even if you do not explicitly create them in your code.

{ Hands-On }

Timers and Implicit Thread Creation

Thread Creation

We can explicitly create a Thread in our code by:

- Extending the Thread class
- Implementing the Runnable interface

The second is the proper way to do it, we are not really extending the thread behaviour but simply giving it something to run, while retaining the possibility for extending from something else.

{ Hands-On }

Explicit Thread Creation

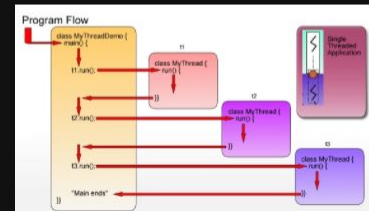
Thread Synchronization

There are times when one thread might want to wait for another thread to finish its execution or for some specific condition to occur

Loops are expensive and will eat up all the available CPU

We can throw a `Thread.sleep()` in the loop, which will ease the CPU by blocking the thread, but there is no way of knowing for sure how much time we should wait. There has to be another way to block those Threads...

Locks provide the proper way to achieve thread synchronization, but for now let's have a look at the `Thread join` method



{ Hands-On }

Thread-Sync using Join

{ Exercise }

Concurrent web server

Thread Safety

Code that executes and behaves correctly when accessed simultaneously by multiple threads is called thread safe

Concurrent programming errors tend to fall into one of these three categories:

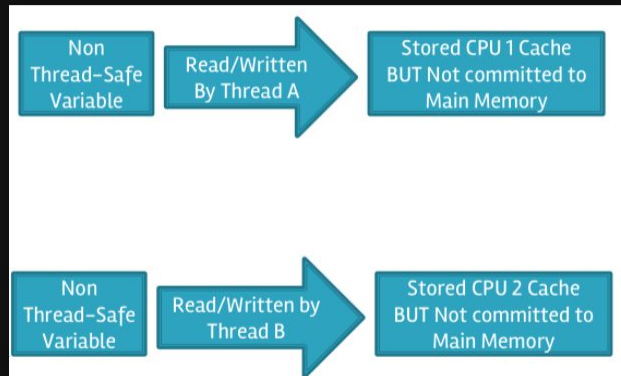
1. visibility
2. atomicity
3. ordering



Visibility

Memory Reads and Writes are slow and expensive operations compared to CPU internal registers. Modern CPUs introduced L1/L2 caches to speed things up.

Variables are not always read from or written to memory, but to the cache of the specific CPU the thread is executing on.



Overcoming Visibility Errors

```
// thread unsafe class
public class IDFactory {

    private long currentId = 0L;

    public long getNextId() {

        // retrieves and writes from cache
        return currentId++;

    }

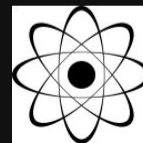
}
```

Declare the variable volatile

Volatile variables will never be cached, all reads and writes will go straight to main memory.

Atomicity

An operation (or set of operations) is atomic if it appears to occur instantaneously to the rest of the system.



Atomic operations commonly have a succeed-or-fail definition; they either successfully change the state of the system, or have no apparent effect.

Non-atomic operations in concurrent programming can cause sporadic and extraneous behaviour, which by nature will likely be hard to reproduce and debug.

Atomicity

```
// thread unsafe class
public class IDFactory {

    volatile private long currentId = 0L;

    public long getNextId() {

        // How many operations are involved in incrementing a long?
        return currentId++;

    }

}
```

- Java primitives are not thread safe
- long and double require two memory operations write or read
- Context switch can occur in the middle of these operations
- Volatile fixes atomicity of reading/writing primitives since Java 5

Atomicity

```
// is this class thread safe?  
public class IDFactory {  
  
    volatile private long currentId = 0;  
  
    public long getNextId() {  
  
        // How many operations are involved here?  
        return currentId++;  
  
    }  
}
```

- Reading, writing and incrementing are distinct operations
- Context switch can occur in the middle of these operations

Atomic Variables

The `java.util.concurrent.atomic` package defines classes that support atomic operations on single variables

All classes have get and set methods that work like reads and writes on volatile variables

```
public class IDFactory {  
    private AtomicLong currentId = new AtomicLong(0);  
    public long getNextId() {  
        return currentId.getAndIncrement();  
    }  
}
```

Atomicity

Atomic Variables solve atomicity problem for isolated variables, but we need to watch for other types of Atomicity issues

```
public void withdrawMoney(int money) {  
    // CHECK  
    if (balance >= money) {  
        // ACT  
        balance -= money;  
    }  
}
```

Watch out for CHECK-THEN-ACT type of logic, everything can happen between checking and acting!

Ordering

The Java compiler can change the order of execution if it believes the outcome will not change...

```
//Code of Thread 1
operationResult = executeVeryComplexComputation();
done = true;

//Code of Thread 2
if (done) {

    // This could go wrong...
    System.out.println(operationResult);

}
```

This compiler assumption does not hold true in multithreading

Solving Concurrency Issues

How do we solve them?

- L1/L2 Caching is solved using Volatile or Locks
- Atomic Memory Reads/Writes is solved using Volatile (for Java > 5) or Locks
- Atomic Check-Then-Act is solved using Locks
- Code Reordering is solved using Locks

Either we learn to cope with concurrency issues or we have to work with immutable objects only...

Immutability

Immutable objects can help us make our objects thread safe.

The final keyword can be used to help enforce immutability:

- a final class cannot be subclassed
- a final method cannot be overridden or hidden by subclasses
- a final variable can only be initialized once

```
// Immutable counter
final class Counter {

    final int count;

    public Counter(int value) {
        this.count = value;
    }

    public int getCount() {
        return count;
    }

    public Counter increment() {
        return new Counter(count + 1);
    }
}
```

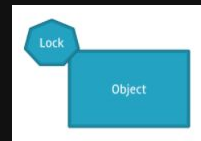
Intrinsic Locks

Java provides a built-in locking mechanism, the synchronized block



The synchronized block has two parts:

1. A reference to an object that will serve as a lock
2. A block of code to be guarded by the lock



```
synchronized (lock) {  
    // Access or modify shared state guarded by lock  
}
```

Synchronized Blocks

Threads acquire a lock when entering a synchronized block

Threads release a lock when exiting the synchronized block

Only one single thread can hold the lock

If a thread attempts to acquire a lock that is already taken, it will block until that lock is released

```
// thread safe class
public class IDFactory {

    // Shared mutable state is guarded by a lock on the object instance
    private long currentId = 0L;

    public long getNextId() {
        synchronized(this) {
            return currentId++;
        }
    }
}
```

Synchronized Methods

Shorthand for synchronizing all the code inside a method with a lock on the object instance

```
// thread safe class
public class IDFactory {

    // Shared mutable state is guarded by a lock on the object instance
    private long currentId = 0L;

    public synchronized long getNextId() {
        return currentId++;
    }
}
```

Thread Synchronization with Guarded Blocks

Efficient way to coordinate actions between Threads

```
public synchronized void guardedHappiness() {  
    // loop until happiness is achieved  
    while(!happy) {  
        try {  
            wait(); // release the lock and block until we get interrupted  
        } catch (InterruptedException e) {}  
    }  
    System.out.println("Happiness has been achieved!");  
}
```

Always invoke wait inside a loop that tests for the condition being waited for.

```
public synchronized void notifyHappiness() {  
    happy = true;  
    notifyAll(); // notify all threads that blocked while using the same lock  
}
```

Wait / Notify

wait, notify and notifyAll are methods inherited from Object

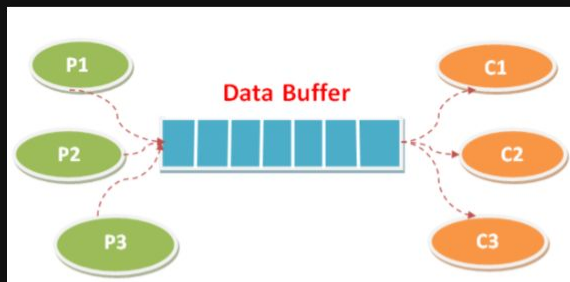
Invoking them on different objects will cause an
IllegalMonitorStateException

notifyAll will unblock all threads and then let them fight for the lock

notify unblocks only one, but each thread needs to remember to
unlock the next one

The Producer/Consumer Problem

Classic example of a multi-process synchronization problem.
The problem describes two processes, the producer and the consumer, who share a common, fixed-size buffer used as a queue.



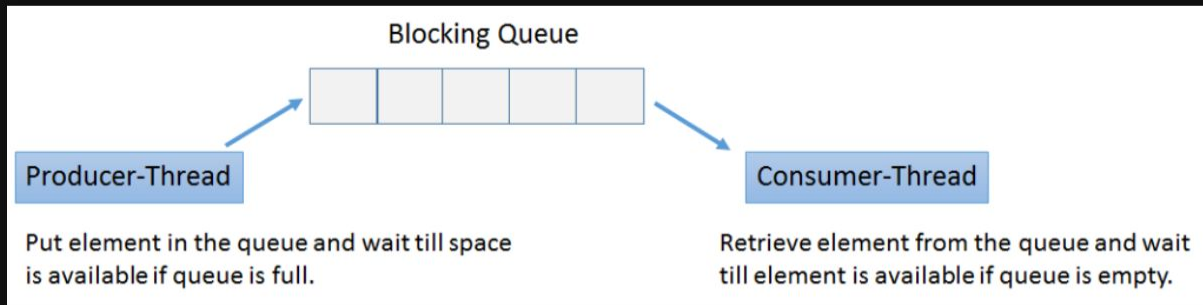
The producer's job is to generate data, put it into the buffer, and start again.

At the same time, the consumer is consuming the data, removing it from the buffer, one piece at a time.

The problem is to make sure that the producer won't try to add data into the buffer if it's full and that the consumer won't try to remove data from an empty buffer.

{ Exercise }

Producer/Consumer using a Blocking Queue



Extra features

Producer alerts when it has left the queue full

Consumer alerts when it has left the queue empty

Synchronized Collections

State is encapsulated and all public methods are synchronized. This makes all operations on these classes atomic at the cost of poor concurrency, which can affect performance

The synchronized collection classes include:

- HashTable - similar to HashMap
- Vector - similar to ArrayList
- Synchronized Wrapper Classes - created by the Collections.synchronized factory methods

Concurrent Collections

Designed for concurrent access from multiple threads, replacing synchronized collections with concurrent collections can offer dramatic scalability improvements

The concurrent collection classes include:

- ConcurrentHashMap - a replacement for synchronized hash based Map implementations
- CopyOnWriteArrayList - a replacement for synchronized List implementations where traversal is the dominant operation

The Wrong Way

Concurrency is hard, can't I just make everything volatile and synchronized?

You can, but...

- That is worse than having a single thread
- You can have a deadlock if you miss something



Thread Pools and Executors

Let's go back to our concurrent web server

```
private void dispatch(ServerSocket bindSocket) {  
    while (true) {  
        // accepts client connections and instantiates client dispatchers  
        ClientDispatcher clientDispatcher = new ClientDispatcher(bindSocket.accept());  
  
        // launch the client thread  
        Thread clientThread = new Thread(clientDispatcher);  
        clientThread.start();  
    }  
}
```

Creating a Thread per Request

Your single threaded Web Server worked but it performed poorly because it could only handle one request at a time

Executing each request in its own thread was an improvement over sequential execution because it enabled the main loop to resume waiting for the next incoming connection more quickly.

But, is it safe?...



{ Hands On }

The ten thousand requests Hell or ...

How my web-server broke down

What just happened?



Creating a new thread for each request can consume significant computing resources.

There are limits on how many threads can be created, imposed by both the JVM and the OS.

When you hit this limit, the most likely result is an `OutOfMemoryError`.

So, no, it is not safe at all

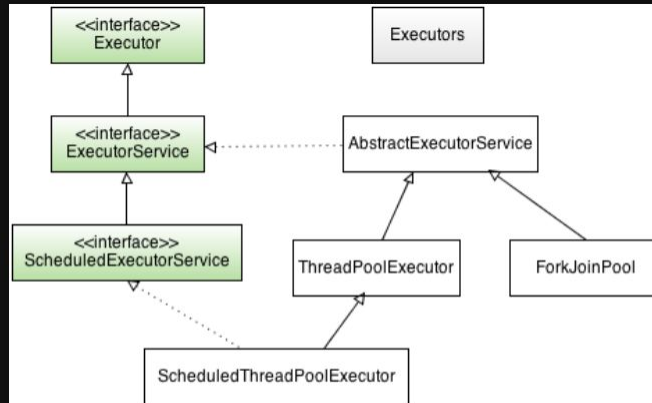
Having nothing but the rate at which users throw HTTP requests at your server to limit thread creation, makes you vulnerable to a malicious user or enough ordinary ones.



The Executor Framework

java.util.concurrent provides a flexible thread pool implementation.

A thread pool is a managed collection of threads that are available to perform tasks.



Executor Interface

The Executor interface provides a standard means of decoupling task submission from task execution, describing tasks with Runnable.

```
public interface Executor {  
  
    /**  
     * Executes the given command at some time in the future. The command  
     * may execute in a new thread, in a pooled thread, or in the calling  
     * thread, at the discretion of the implementation.  
     */  
    void execute(Runnable command);  
}
```

Why should I use them?

Reusing an existing thread reduces the costs of creating a new one for each request

Since most of the time, the worker thread is already there, task execution will not be delayed, improving responsiveness.

It gives you access to other actions such as tuning, management, monitoring, logging, or error reporting.

ExecutorService

The `ExecutorService` interface extends `Executor` and provides multiple convenience methods for the lifecycle management of the `Executor` and submission of tasks.

An `ExecutorService` can be shut down, which will cause it to reject new tasks



Creating a new thread pool

You can create a thread pool by calling one of the static factory methods in Executors

Fixed Thread Pool

```
// Allocate a Pool of 4 fixed threads
ExecutorService fixedPool = Executors.newFixedThreadPool(4);

// submit 10 tasks to be executed
for(int i = 0; i < 10; i++){
    fixedPool.submit(new Runnable());
}

// shut down the executor after all submitted tasks are finished
fixedPool.shutdown();
```

A fixed-size thread pool creates threads as tasks are submitted, up to the maximum pool size, and then attempts to keep the pool size constant (adding new threads if a thread dies due to an unexpected Exception).

Cached Thread Pool

```
// Allocate a Pool of an undefined number of threads
ExecutorService cachedPool = Executors.newCachedThreadPool();

// submit 10 tasks to be executed
for(int i = 0; i < 10; i++){
    cachedPool.submit(new Runnable());
}

// shut down the executor immediately, even if the submitted tasks haven't finished its execution
cachedPool.shutdownNow();
```

A much more flexible pool, will terminate idle threads when the current size of the pool exceeds the demand for processing, and add new ones when demand increases. Does not impose a size limit to the pool.

Single Thread Executor

```
// Create an executor for a single thread
ExecutorService singleExecutor = Executors.newSingleThreadExecutor();

// submit 10 tasks to be executed sequentially
for(int i = 0; i < 10; i++){
    singleExecutor.submit(new Runnable());
}

// shut down the executor after all submitted tasks are finished
singleExecutor.shutdown();
```

Creates a single worker thread to process tasks, replacing it if it dies unexpectedly. Tasks are guaranteed to be processed sequentially according to the order imposed by the task queue (FIFO, LIFO, priority order).

{ Exercise }

The 10000 requests Hell or ...

How my web-server stood up **STRONG!**

{ Exercise }

Concurrent chat server with support for commands